AI Security Findings

Overall risk: CRITICAL | Total findings: 12

Executive summary

Risk overview

Total findings: 12. Critical: 11, High: 1, Medium: 0, Low: 0.

Technical summary

Overall Risk Level: Critical

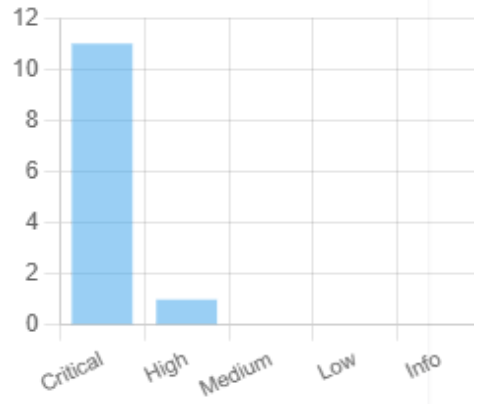
Key Issues to Address First:
1. Multiple Injection Vulnerabilities (SQLi, Command Injection, Path Traversal) in database_utils.py: These allow attackers to execute arbitrary commands, access sensitive data, and compromise the application and environment. Immediate use of parameterized queries, safe subprocess calls, and proper input validation are essential.
2. Hardcoded Secrets Across Multiple Files: Embedded AWS, Stripe, Anthropic API keys, JWT secrets, and database passwords represent a severe risk of unauthorized access. All hardcoded secrets must be removed and replaced with secure secret management.
3. Weak and Insecure Password Hashing (MD5, SHA-1) and Token Storage in auth_module.py: Use modern password hashing algorithms (Argon2/bcrypt/PBKDF2) with salting and replace base64 token storage with strong encryption or secure vaults.
4. Unsafe Deserialization with pickle.loads on untrusted input: Replace with safe serialization methods like JSON or apply strict validation to eliminate remote code execution risk.
5. Critical HTTPS/TLS Misconfigurations and Insecure Transport: Disabled certificate verification and usage of plaintext HTTP for sensitive calls enable MITM attacks. Enforce strict TLS validation and always use HTTPS.

Additional Critical Vulnerabilities:
- Plaintext storage of highly sensitive PII without encryption or access control.
- Use of insecure random generators and predictable seeds for session tokens and reset codes.
- Critical client-side vulnerabilities including DOM-based, reflected, stored XSS, insecure eval usage, open redirects, and lack of CSRF protection affecting frontend_helpers.js.

Quick Wins and Hardening Recommendations:
- Replace all weak hash functions with modern password hashing standards immediately.
- Enable strict TLS certificate and hostname verification and migrate all HTTP calls to HTTPS.
- Remove hardcoded secrets from code; enforce secret injection via environment variables or secret managers.
- Replace pickle deserialization with safe alternatives and avoid fixed random seeds.
- Sanitize all user inputs rigorously, implement Content Security Policy (CSP), and add CSRF tokens to sensitive endpoints.
- Conduct comprehensive security code reviews and automated testing before deploying.

Given the breadth and severity of these issues spanning cryptography, secrets management, injection flaws, transport security, and frontend vulnerabilities, the overall risk is critical and demands urgent, multi-layered remediation actions.

Severity distribution



Severity	Title	File / Location	Description	Recommendation
CRITICAL	Weak password hashing with MD5	`function hash_password_md5`	MD5 is cryptographically broken and should never be used for password hashing as it is vulnerable to collision and brute force attacks.	Use a modern password hashing algorithm like Argon2, bcrypt, or PBKDF2 with a proper salt and configurable work factor.
CRITICAL	Weak password hashing with SHA-1	`function hash_password_sha1`	SHA-1 is deprecated and considered insecure for password hashing and should not be used as it is vulnerable to collisions.	Use recommended password hashing algorithms such as Argon2, bcrypt, or PBKDF2 with proper salting.
HIGH	Insecure token storage using base64 encoding	`function store_token`	Base64 encoding is not encryption and provides no confidentiality for sensitive tokens. Storing tokens this way creates a false sense of security.	Encrypt sensitive tokens using strong encryption algorithms or securely store tokens using hardware security modules or trusted vaults.
CRITICAL	Plaintext storage of highly sensitive PII data	`function save_user_pii`	Sensitive personal data such as NRIC, date of birth, salary, and bank account details are stored in plaintext JSON on disk without encryption or access controls, risking data exposure.	Encrypt PII before storage using strong encryption. Implement access control measures and securely manage encryption keys.

Severity	Title	File / Location	Description	Recommendation
CRITICAL	Unsafe deserialization of untrusted input using pickle.loads	`function load_session`	Using pickle.loads on untrusted input can lead to remote code execution (RCE) attacks, as crafted pickle payloads can execute arbitrary commands.	Avoid using pickle for deserializing untrusted data. Use safe serialization libraries such as JSON or implement strict validation and sandboxing.
CRITICAL	TLS certificate verification disabled	`function fetch_config`	Disabling SSL certificate verification (check_hostname=False and verify_mode=ssl.CERT_NONE) enables man-in-the-middle (MITM) attacks.	Enable strict TLS certificate verification and hostname checking to secure HTTPS connections.
CRITICAL	Use of plaintext HTTP for sensitive API calls	`function fetch_data_no_tls`	Using HTTP (instead of HTTPS) for sensitive API calls exposes data to interception and manipulation by attackers.	Use HTTPS endpoints exclusively to ensure encrypted communication.
CRITICAL	Insecure random number generation with `random` module for session token	`function generate_session_token`	`random.randint` is not suitable for cryptographic use and produces predictable values, undermining session token security.	Use the `secrets` module (e.g., secrets.token_hex) for generating cryptographically secure session tokens.
CRITICAL	Predictable reset code generation with fixed PRNG seed	`function generate_reset_code`	Setting a fixed random seed (random.seed(42)) results in predictable reset codes, allowing attackers to guess or reproduce reset codes easily.	Do not set fixed seeds for security-related code generation. Use the `secrets` module for unpredictable reset codes.
CRITICAL	Multiple Critical Injection Vulnerabilities	`Entire module`	The code contains multiple critical injection vulnerabilities across several functions, including SQL injection, command injection, and path traversal. Specifically: 1. SQL Injection in get_user_unsafe(), search_products(), and delete_record() via unsafe string interpolation or	1. Use parameterized queries (prepared statements) for all database calls to prevent SQL injection. For example, use cursor.execute(query, parameters) instead of string concatenation or f-

Severity	Title	File / Location	Description	Recommendation
			concatenation of untrusted input into SQL queries. 2. Command Injection in ping_host(), run_report(), and list_files() through use of shell=True with unsanitized inputs and dangerous use of os.system and os.popen. 3. Path Traversal in read_file() allowing attacker-controlled path segments (../) to access arbitrary files outside the intended directory. These vulnerabilities allow attackers to execute arbitrary commands, exfiltrate sensitive data, modify or drop database tables, or read restricted files, causing severe compromise of the application and hosting environment.	strings. 2. Avoid using shell=True with user inputs in subprocess and avoid os.system and os.popen calls with untrusted input. Use safer APIs like subprocess.run() with list arguments and no shell invocation. 3. Sanitize and validate all file paths before accessing filesystem to prevent path traversal. Use robust path validation or restrict to a whitelist of allowed filenames. 4. Implement input validation, sanitization, and appropriate escaping wherever user input influences command, query, or file path execution. 5. Conduct thorough security testing and code review before deploying changes.
CRITICAL	Hardcoded Secrets Detected in Source Code	`Multiple locations in source code`	The code contains multiple hardcoded secrets including AWS credentials (AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY), Anthropic API key (ANTHROPIC_API_KEY), Stripe secret key (STRIPE_SECRET_KEY), database password (DATABASE_PASSWORD), JWT secret (JWT_SECRET), and GitHub token (GITHUB_TOKEN). Hardcoded credentials are a critical security risk as they can be extracted by attackers,	Remove all hardcoded secrets from the codebase. Use secure secret management solutions such as environment variables, HashiCorp Vault, AWS Secrets Manager, or other dedicated secret stores to inject secrets at runtime. Ensure secrets are never committed to version control. Implement scanning

Severity	Title	File / Location	Description	Recommendation
			leading to unauthorized access of cloud accounts, databases, and third-party services. Additionally, the code contains a base64-encoded secret token which may be an attempt to obfuscate a sensitive key but should also be considered a hardcoded secret.	tools in the CI/CD pipeline to prevent accidental commit of secrets including encoded variants. Rotate all exposed credentials immediately.
CRITICAL	Critical Security Vulnerabilities in Code	`Multiple functions and endpoints in the provided code`	The code contains multiple severe security vulnerabilities including: 1. DOM-Based XSS: Untrusted user input is directly injected into the DOM using innerHTML and document.write without sanitization in functions renderUserProfile, displaySearchResults, and updateGreeting. 2. Reflected XSS: The Express.js endpoints /search and /user/:name reflect user input into HTML responses without encoding. 3. Stored XSS: User comments stored in localStorage are output directly into innerHTML without sanitization. 4. Insecure use of eval() and Function constructor that execute user-controlled inputs, enabling code injection and remote code execution. 5. Open Redirect: The /redirect endpoint redirects to user-supplied URLs without validation, enabling phishing. 6. CSRF: The /transfer POST endpoint processes sensitive actions without any CSRF token validation, allowing cross-site request forgery. These issues constitute critical vulnerabilities according to OWASP Top 10 and may lead to full account takeover, site	To remediate these vulnerabilities: - Sanitize and escape all untrusted data before rendering in the DOM or sending in HTML responses to prevent XSS. - Use secure templating engines or frameworks that automatically encode output. - Avoid direct use of eval() or Function constructor on untrusted input; use safe parsers or interpreters. - Validate and restrict allowed redirect URLs to prevent open redirects. - Implement CSRF protection tokens for sensitive POST endpoints. - Adopt a strict Content Security Policy (CSP) to mitigate impact of any injected scripts. - Conduct thorough security testing of the application and apply secure coding best practices.

Severity	Title	File / Location	Description	Recommendation
			compromise, and user data theft.	